セルフオーダーシステム 制作③ 注文確定

Session のカートを、注文履歴として SQL Server に保存する

Visual Studio 2022 ／ .NET 8　　Version 2.2

この回で実現すること

カート段階（制作②）	注文確定段階（制作③）
Session に商品・数量を一時保存	DB に注文履歴を永続保存
商品を追加・変更・削除できる	Order と OrderItem を登録する
カートの価格は登録時点の情報	確定直前に DB の商品状態・価格を再確認

画面とデータの流れ

メニュー → カート（Session） → 注文確認（テーブル番号入力） → DB 再確認 → Orders／OrderItems を保存 → カートを空にする → 注文完了 → 注文状況

到達目標

- Orders／OrderItems の「親と明細」の関係を説明できる
- 既存 AppDbContext に注文の設定を安全に追加できる
- async／await を使って DB 保存を完了させられる
- 注文確定後に注文状況画面を表示できる

教科書との接続

教科書	制作③で使う場面
4.5 列挙型	OrderStatus の状態管理
5.1 プロパティ／7.3 集計	Order・OrderItem・Total
13 章 LINQ／14.3 非同期処理	商品検索と SaveChangesAsync
15.4 依存性の注入（後の授業）	OrdersController が CartService を受け取る

【復習】 教科書 14 章の非同期処理は既習です。制作③では、DB の検索や保存に実際に使用します

STEP 0　制作②の状態を確認する

作業前にバックアップまたは Git コミットを行います。すでに動作しているメニュー表示とカート操作は削除しません。

確認する場所	確認内容
Models/Product.cs	Id、Name、Price、IsActive、IsSoldOut が存在する
Models/CartItem.cs	ProductId、ProductName、UnitPrice、Quantity、LineTotal がある
Services/CartService.cs	GetItems()、Clear()が存在する
Program.cs	AddDbContext、AddSession、AddHttpContextAccessor、AddScoped<CartService>がある
Views/_ViewImports.cshtml	@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers がある
SQL Server LocalDB	制作①で作った SelfOrderSystemDb へ接続できる

【重複防止】 Order.cs／OrderItem.cs／OrderStatus.cs／OrdersController.cs が既に存在する場合は新規作成せず、内容を比較してください

STEP 1　注文の状態を定義する

新規作成：Models/OrderStatus.cs（既にある場合は内容の照合のみ）

Models/OrderStatus.cs

```
namespace SelfOrderSystem.Models;

public enum OrderStatus          // 注文の状態を 4 種類で表す
{
    New = 0,                    // 新規注文
    Accepted = 1,               // 受付済み
    Cooking = 2,                // 調理中
    Served = 3                  // 提供済み
}
```

enum は状態を数値だけでなく名前で扱う仕組みです。注文直後は New、店舗が受け付けたら Accepted になります。

STEP 1（続き）画面用の日本語に変換する

新規作成：Models/OrderStatusExtensions.cs（既存なら再作成しない）

Models/OrderStatusExtensions.cs（後の店舗受付編でも利用）

```
namespace SelfOrderSystem.Models;

public static class OrderStatusExtensions
{
    public static string ToJapanese(this OrderStatus status)
    {
        return status switch
        {
            OrderStatus.New => "新規注文",
            OrderStatus.Accepted => "受付済み",
            OrderStatus.Cooking => "調理中",
            OrderStatus.Served => "提供済み",
            _ => "不明"
        };
    }

    public static string NextActionLabel(this OrderStatus status)
    {
        return status switch
        {
            OrderStatus.New => "受付する",
            OrderStatus.Accepted => "調理開始",
            OrderStatus.Cooking => "提供完了",
            _ => "完了"
        };
    }
}
```

なぜ「拡張メソッド」なのか

OrderStatus は enum なので、その中にメソッドは直接追加できません。拡張メソッドを使うと、OrderStatus.New.ToJapanese() のように呼び出せます。

記述	意味
static class	拡張メソッドを定義するクラス
this OrderStatus status	OrderStatus 型から呼び出せるようにする指定
status switch	状態ごとに表示文字列を切り替える

【未習範囲への配慮】 拡張メソッドは教科書 16.4 節で学ぶ内容です。今回は使い方だけ確認し、仕組みの詳細は後で扱います

STEP 2　注文を表す Model を作る

新規作成：Models/Order.cs、Models/OrderItem.cs　※同名クラスがある場合は照合・不足項目のみ追加

Models/Order.cs（1 件の注文全体）

```
using System.ComponentModel.DataAnnotations;

namespace SelfOrderSystem.Models;

public class Order                 // 注文全体を表す親 Model
{
    public int Id { get; set; }   // DB 内の注文 ID
    public Guid PublicId { get; set; } = Guid.NewGuid();

    [Range(1, 999)]
    public int TableNumber { get; set; }  // テーブル番号
    public DateTimeOffset OrderedAtUtc { get; set; } // UTC の注文日時
    public OrderStatus Status { get; set; } = OrderStatus.New;
    public List<OrderItem> Items { get; set; } = new();
}
```

Models/OrderItem.cs（注文商品 1 種類分）

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SelfOrderSystem.Models;

public class OrderItem             // 注文に含まれる商品 1 行
{
    public int Id { get; set; }
    public int OrderId { get; set; }    // 親注文への外部キー
    public int ProductId { get; set; }  // 商品への外部キー
    [Range(1, 99)]
    public int Quantity { get; set; }
    [Column(TypeName = "decimal(10,0)")]
    public decimal UnitPrice { get; set; } // 注文時の確定単価
    public Order? Order { get; set; }
    public Product? Product { get; set; }
    [NotMapped]
    public decimal LineTotal => UnitPrice * Quantity;
}
```

DB の関係：Orders.Id（1 件） → OrderItems.OrderId（複数件）／ OrderItems.ProductId → Products.Id

【設計の要点】 Order.Id は DB 用の連番、PublicId は画面 URL 用の識別子です。注文後に商品価格が変わっても過去の金額は OrderItem.UnitPrice に残ります

STEP 3　AppDbContext へ注文テーブルを登録する

編集：Data/AppDbContext.cs。制作①の AppDbContext を保ち、Orders・OrderItems を追加します。

OnModelCreating が無い場合：下記の完成形にする

Data/AppDbContext.cs（制作①の構成に追記した見本）

```
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Models;

namespace SelfOrderSystem.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Order>()
            .HasIndex(o => o.PublicId)
            .IsUnique();                // 公開 ID の重複を防止

        modelBuilder.Entity<OrderItem>()
            .HasOne(i => i.Product)
            .WithMany()
            .HasForeignKey(i => i.ProductId)
            .OnDelete(DeleteBehavior.Restrict); // 注文履歴を保護
    }
}
```

【重要】 すでに OnModelCreating がある場合はメソッド全体を追加せず、既存メソッドの中へ HasIndex と HasOne の設定だけを追記します。既存の処理は削除しません

STEP 4　Migration で DB を更新する

操作場所：Visual Studio 2022 →［ツール］→［NuGet パッケージ マネージャー］→［パッケージ マネージャー コンソール］

まず現在の DB と Migration を確認

- ソリューションをビルドしてエラー0 件を確認
- 「既定のプロジェクト」を SelfOrderSystem に指定
- SQL Server オブジェクト エクスプローラーで Orders・OrderItems の有無を確認
- Get-Migration で過去の Migration を確認

注文関連の Migration が未作成の場合に限り実行

```
Add-Migration AddOrders
Update-Database
```

Add-Migration で生成されたファイルの Up()を確認してから Update-Database を実行します。期待する変更は Orders と OrderItems および関連するキー・インデックスです。

状況	実施すること
注文テーブルがなく、対応 Migration もない	Add-Migration → 内容確認 → Update-Database
注文 Migration は存在し、DB だけ未適用	Update-Database のみ実行
すでに注文テーブルがあり適用済み	同じ Migration を追加しない
生成内容に既存テーブル削除や想定外の変更	Update-Database は止めて教員へ確認

STEP 5　注文確認用 ViewModel を作る

新規作成：ViewModels/CheckoutViewModel.cs

ViewModels/CheckoutViewModel.cs

```
using System.ComponentModel.DataAnnotations;
using SelfOrderSystem.Models;

namespace SelfOrderSystem.ViewModels;

public class CheckoutViewModel
{
    [Display(Name = "テーブル番号")]
    [Range(1, 999, ErrorMessage = "1〜999 を入力してください")]
    public int TableNumber { get; set; }
    public List<CartItem> Items { get; set; } = new();
    public decimal Total => Items.Sum(x => x.LineTotal);
}
```

STEP 6　OrdersController を作る（1／3）

新規作成：Controllers/OrdersController.cs。次の 3 ページのコードを順につなげて 1 つのファイルにします。

このページでは using、コンストラクタ、注文確認画面の表示まで書きます。クラスの閉じ括弧は最後のページにあります。

OrdersController.cs　前半：クラス開始〜Checkout()

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Data;
using SelfOrderSystem.Models;
using SelfOrderSystem.Services;
using SelfOrderSystem.ViewModels;

namespace SelfOrderSystem.Controllers;

public class OrdersController : Controller
{
    private readonly AppDbContext _db;      // 商品と注文の DB 操作
    private readonly CartService _cart;    // Session 内のカート操作

    public OrdersController(AppDbContext db, CartService cart)
    {
        _db = db;
        _cart = cart;
    }

    [HttpGet]
    public IActionResult Checkout()
    {
        var items = _cart.GetItems();     // Session からカートを読む
        if (items.Count == 0)
        {
            return RedirectToAction("Index", "Menu");
        }

        return View(new CheckoutViewModel { Items = items });
    }
```

処理の意味

コード	役割
OrdersController(AppDbContext db, CartService cart)	DI で DB 操作とカート操作を受け取る
_cart.GetItems()	Session に保存した商品の一覧を取得
View(new CheckoutViewModel { Items = items })	注文確認画面へカートの内容を渡す

【貼り付け注意】 このページ末尾では OrdersController のクラスはまだ閉じません。次の「Place()」を同じクラスの中に追加します

STEP 6　OrdersController を作る（2／3）

前ページの Checkout()の直後に、次の Place()を追加します。これは 1 つの完成したメソッドです。

OrdersController.cs　続き：Place()の全体

```
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Place(CheckoutViewModel model)
    {
        var cartItems = _cart.GetItems(); // Session を再取得
        if (cartItems.Count == 0)
            ModelState.AddModelError("", "カートが空です");

        var ids = cartItems.Select(x => x.ProductId).Distinct().ToList();
        var products = await _db.Products.AsNoTracking()
            .Where(p => ids.Contains(p.Id) && p.IsActive)
            .ToDictionaryAsync(p => p.Id); // 現在の DB から商品情報を取得

        foreach (var item in cartItems)
        {
            if (!products.TryGetValue(item.ProductId, out var product))
            {
                ModelState.AddModelError("", "注文できない商品があります");
                continue;
            }
            if (product.IsSoldOut)
                ModelState.AddModelError("", $"{product.Name}は売り切れです");
            if (item.Quantity < 1 || item.Quantity > 99)
                ModelState.AddModelError("", "数量は 1〜99 にしてください");
            if (product.Price != item.UnitPrice)
                ModelState.AddModelError("", $"{product.Name}の価格が変更されました");
        }

        if (!ModelState.IsValid)
        {
            model.Items = cartItems;     // 再表示用の一覧を戻す
            return View("Checkout", model);
        }

        var order = new Order
        {
            PublicId = Guid.NewGuid(),
            TableNumber = model.TableNumber,
            OrderedAtUtc = DateTimeOffset.UtcNow,
            Status = OrderStatus.New
        };
        foreach (var item in cartItems)
        {
            var product = products[item.ProductId];
            order.Items.Add(new OrderItem
            {
                ProductId = product.Id,
                Quantity = item.Quantity,
                UnitPrice = product.Price // 注文確定時の DB 価格を保存
            });
        }

        _db.Orders.Add(order);           // 注文と明細を登録対象にする
        await _db.SaveChangesAsync();   // DB 保存の完了を待つ
        _cart.Clear();                  // 保存成功後にカートを空にする
        return RedirectToAction(nameof(Complete),
            new { id = order.PublicId });
    }
```

STEP 6　OrdersController を作る（3／3）

Place()の後に注文完了・状況画面の処理を追加します。最後の } で OrdersController クラスを閉じます。

OrdersController.cs　末尾：Complete()・Status()・共通取得処理

```
    [HttpGet]
    public async Task<IActionResult> Complete(Guid id)
    {
        var order = await LoadPublicOrderAsync(id);
        return order == null ? NotFound() : View(order);
    }

    [HttpGet]
    public async Task<IActionResult> Status(Guid id)
    {
        var order = await LoadPublicOrderAsync(id);
        return order == null ? NotFound() : View(order);
    }

    private Task<Order?> LoadPublicOrderAsync(Guid publicId)
    {
        return _db.Orders.AsNoTracking()
            .Include(o => o.Items)
            .ThenInclude(i => i.Product)
            .SingleOrDefaultAsync(o => o.PublicId == publicId);
    }
}
```

非同期処理の復習（教科書 14.3）

記述	何を待っているか
await …ToDictionaryAsync()	商品情報を DB から取得する完了
await _db.SaveChangesAsync()	Orders と OrderItems の保存完了
await LoadPublicOrderAsync()	注文履歴 1 件の取得完了
Task<Order?>	後から Order または null が返る処理

【授業のポイント】 await は DB 処理の完了を待ちますが、待ち時間中にスレッドを占有し続けるものではありません。EF Core の非同期 I/O を利用するのが目的です

STEP 7　注文確認画面を作る

新規作成：Views/Orders/Checkout.cshtml　※Orders フォルダーがなければ作成

Views/Orders/Checkout.cshtml（完成コード）

```
@model SelfOrderSystem.ViewModels.CheckoutViewModel
@{
    ViewData["Title"] = "注文確認";
}
<h1 class="h3">注文内容の確認</h1>
<div asp-validation-summary="ModelOnly" class="text-danger"></div>

@foreach (var item in Model.Items)
{
    <div class="d-flex justify-content-between border-bottom py-2">
        <span>@item.ProductName × @item.Quantity</span>
        <span>@item.LineTotal.ToString("N0") 円</span>
    </div>
}
<div class="text-end fs-4 fw-bold my-3">
    合計 @Model.Total.ToString("N0") 円
</div>

<form asp-action="Place" method="post">
    <div class="mb-3">
        <label asp-for="TableNumber" class="form-label"></label>
        <input asp-for="TableNumber" class="form-control" />
        <span asp-validation-for="TableNumber" class="text-danger"></span>
    </div>
    <button class="btn btn-success btn-lg w-100" type="submit">
        注文を確定する
    </button>
</form>
<a asp-controller="Cart" asp-action="Index"
   class="btn btn-outline-secondary mt-3">カートに戻る</a>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

見てほしいポイント

- @model で CheckoutViewModel を受け取る
- asp-action="Place" と method="post" で注文確定処理を呼び出す
- asp-validation-for と asp-validation-summary で入力エラー・商品検証エラーを表示する
- カートに戻るリンクを追加し、価格変更時に商品を入れ直せるようにする

【価格変更時】「価格が変更されました」と表示された場合は、カートから対象商品を削除し、メニューから改めて追加して確認してください

STEP 8　注文完了画面と注文状況画面

新規作成：Views/Orders/Complete.cshtml と Views/Orders/Status.cshtml

Views/Orders/Complete.cshtml

```
@model SelfOrderSystem.Models.Order
@{
    ViewData["Title"] = "注文完了";
}
<div class="text-center py-4">
    <h1 class="h3">ご注文ありがとうございます</h1>
    <p class="display-6">注文番号 #@Model.Id</p>
    <p>テーブル @Model.TableNumber</p>
    <a class="btn btn-primary" asp-action="Status"
       asp-route-id="@Model.PublicId">注文状況を確認する</a>
</div>
```

Views/Orders/Status.cshtml

```
@using SelfOrderSystem.Models
@model SelfOrderSystem.Models.Order
@{
    ViewData["Title"] = "注文状況";
}
<h1 class="h3">注文 #@Model.Id</h1>
<p>テーブル @Model.TableNumber</p>
<div class="alert alert-dark fs-4 fw-bold">
    現在：@Model.Status.ToJapanese()
</div>
@foreach (var item in Model.Items)
{
    <div class="d-flex justify-content-between border-bottom py-2">
        <span>@item.Product?.Name × @item.Quantity</span>
        <span>@item.LineTotal.ToString("N0") 円</span>
    </div>
}
```

【つまずき防止】 Status.cshtml の @using SelfOrderSystem.Models は、拡張メソッド ToJapanese()を認識させるために必要です。
_ViewImports.cshtml に設定済みなら重複記述は不要です

STEP 9　カートから注文確認へ進むリンク

既存編集：Views/Cart/Index.cshtml。既存の商品一覧・数量変更・削除フォームを残し、一覧の下へ追記します。

Model.Any()でカートが空でない場合だけボタンを表示します。

Views/Cart/Index.cshtml へ追記

```
@if (Model.Any())
{
    <a class="btn btn-success btn-lg w-100"
        asp-controller="Orders" asp-action="Checkout">
        注文内容を確認する
    </a>
}
```

STEP 10　動作確認（順番に試す）

操作	期待結果
メニューで 2 種類の商品を追加	カートに 2 種類が表示される
カート→注文内容を確認する	Checkout に商品・数量・合計が表示される
テーブル番号 0 または 1000 で確定	入力検証エラーになり DB には登録されない
商品を売り切れにして確定	エラーが出て注文を作成しない
商品価格を変えてから確定	価格変更エラーになり DB 登録されない。商品を入れ直して再確認
販売可能な商品を適切な番号で確定	Orders に 1 件、OrderItems に商品種類分の行が追加
完了画面から注文状況へ移動	新規注文と明細が表示される
メニュー・カートを再表示	カートが空になっている

DB の確認

SQL Server オブジェクト エクスプローラーで SelfOrderSystemDb の dbo.Orders、dbo.OrderItems を開き、保存内容を確認します。

【検証上の注意】 価格変更や売り切れの確認は、別ブラウザーまたは DB から商品状態を更新して試験します。完成直前の DB を壊さないよう、テスト後は元の価格・状態へ戻してください

よくあるエラーと対処

症状	主な確認箇所
Order が二重定義されている	同名 Model を新規作成していないか
OnModelCreating に関するエラー	メソッドを 2 つ作っていないか／クラス内に記述したか
Orders テーブルがない	Get-Migration と Update-Database の適用状況
@Model.Status.ToJapanese()でエラー	OrderStatusExtensions と@using があるか
注文確認画面が 404	OrdersController.Checkout()と Views/Orders の場所
Session が取れない	Program.cs の Session 登録・UseSession の位置
注文時に売り切れ・価格変更エラー	最新の商品状態とカート内容を確認し直す
保存できない・DB 接続に失敗	接続文字列と Migration、出力ウィンドウの例外

学習したポイントをまとめよう

確認問題	考えるポイント
なぜカート情報を直接注文テーブルへ移さないのか	商品状態・価格を DB で再確認するため
Order と OrderItem を分ける理由は	1 件の注文に複数の商品明細があるため
なぜ SaveChangesAsync()に await を付けるのか	保存の完了を待ってからカートを消すため
Session を消すのはいつか	DB 保存が成功した後
Controller が CartService を new しない理由は	ASP.NET Core の DI から受け取るため

次の制作④へ

制作④では、OrdersController が担当している「商品の検証・注文生成・DB 保存」を OrderService へ分離します。

DI 入門で学んだ「外部から渡してもらう」仕組みを実際に使います。

【発展】 ここまでのコードは授業用の基本実装です。実店舗への公開には、二重送信防止や注文の閲覧権限などの追加対策が必要です